

Java Collection Framework — Important Methods by Interface

Aim

To explore the Java Collection Framework by implementing and demonstrating the important methods of its core interfaces and classes — Set (HashSet, TreeSet/NavigableSet), Queue and Deque (ArrayDeque), Map (TreeMap/NavigableMap), the legacy classes Vector and Stack, the List implementations LinkedList and ArrayList, and traversal using Iterator and ListIterator.

Question:

Java Collection Framework: Important Methods by **Interface**

Collection Interface

- `add(E e)` - Inserts a specific element into the collection.
- `addAll(Collection<? extends E> c)` - Adds all elements of the specified collection to this collection.
- `remove(Object o)` - Removes a single instance of the specified element.
- `removeAll(Collection<?> c)` - Removes all elements that are contained in the specified collection.
- `contains(Object o)` - Returns true if this collection contains the specified element.
- `containsAll(Collection<?> c)` - Returns true if this collection contains all elements of the specified collection.
- `size()` - Returns the total number of elements in the collection.
- `isEmpty()` - Checks if the collection has no elements.
- `clear()` - Removes all elements from the collection.
- `iterator()` - Returns an iterator over the elements in this collection.

List Interface (Ordered, allows duplicates)

- `add(E e)` - Appends the element to the end of the list.
- `add(int index, E element)` - Inserts the element at the specified position.
- `get(int index)` - Retrieves the element at the specified position.
- `set(int index, E element)` - Replaces the element at the specified position with the new element.
- `remove(int index)` - Removes the element at the specified index position.

- `indexOf(Object o)` - Returns the index of the first occurrence of the element.
- `lastIndexOf(Object o)` - Returns the index of the last occurrence of the element.
- `subList(int fromIndex, int toIndex)` - Returns a view of the portion of this list between the specified indexes.
- `sort(Comparator<? super E> c)` - Sorts the list according to the order induced by the specified Comparator.

● **Set Interface (Unordered, unique elements)**

- `add(E e)` - Adds the element only if it is not already present.
- `remove(Object o)` - Removes the specified element if present.
- `contains(Object o)` - Checks if the set contains the specified element.
- `size()` - Returns the number of elements (cardinality) of the set.
- `isEmpty()` - Checks if the set is empty.
- `clear()` - Removes all elements from the set.

☑ **SortedSet Interface (Elements sorted in ascending order)**

- `first()` - Returns the first (lowest) element currently in this set.
- `last()` - Returns the last (highest) element currently in this set.
- `headSet(E toElement)` - Returns a view of the portion of this set whose elements are strictly less than toElement.
- `tailSet(E fromElement)` - Returns a view of the portion of this set whose elements are greater than or equal to fromElement.
- `subSet(E fromElement, E toElement)` - Returns a view of the portion of this set whose elements range from fromElement (inclusive) to toElement (exclusive).
- `comparator()` - Returns the comparator used to order the elements, or null if natural ordering is used.

☐ **NavigableSet Interface (Extended SortedSet with navigation methods)**

- `lower(E e)` - Returns the greatest element strictly less than the given element.
- `floor(E e)` - Returns the greatest element less than or equal to the given element.
- `ceiling(E e)` - Returns the least element greater than or equal to the given element.
- `higher(E e)` - Returns the least element strictly greater than the given element.
- `pollFirst()` - Retrieves and removes the first (lowest) element, or returns null if empty.
- `pollLast()` - Retrieves and removes the last (highest) element, or returns null if empty.

- `descendingSet()` - Returns a reverse order view of the elements contained in this set.

Queue Interface (FIFO - First In First Out)

- `add(E e)` - Inserts the element. Throws an exception if it fails.
- `offer(E e)` - Inserts the element. Returns false if it fails (preferred for capacity-restricted queues).
- `remove()` - Retrieves and removes the head of the queue. Throws an exception if empty.
- `poll()` - Retrieves and removes the head of the queue. Returns null if empty.
- `element()` - Retrieves, but does not remove, the head of the queue. Throws an exception if empty.
- `peek()` - Retrieves, but does not remove, the head of the queue. Returns null if empty.

Deque Interface (Double-Ended Queue)

- `addFirst(E e) / addLast(E e)` - Inserts the element at the front / end of the deque (throws exception if full).
- `offerFirst(E e) / offerLast(E e)` - Inserts the element at the front / end of the deque (returns false if full).
- `removeFirst() / removeLast()` - Retrieves and removes the first / last element (throws exception if empty).
- `pollFirst() / pollLast()` - Retrieves and removes the first / last element (returns null if empty).
- `peekFirst() / peekLast()` - Retrieves, but does not remove, the first / last element (returns null if empty).

Map Interface (Key-Value pairs)

- `put(K key, V value)` - Associates the specified value with the specified key.
- `get(Object key)` - Returns the value to which the specified key is mapped.
- `remove(Object key)` - Removes the mapping for a key if it is present.
- `containsKey(Object key)` - Returns true if this map contains a mapping for the specified key.
- `containsValue(Object value)` - Returns true if this map maps one or more keys to the specified value.
- `keySet()` - Returns a Set view of the keys contained in this map.
- `values()` - Returns a Collection view of the values contained in this map.
- `entrySet()` - Returns a Set view of the key-value mappings contained in this map.
- `size()` - Returns the number of key-value mappings in this map.
- `isEmpty()` - Checks if the map contains no key-value mappings.
- `clear()` - Removes all mappings from this map.

SortedMap Interface (Keys sorted in ascending order)

- `firstKey()` - Returns the first (lowest) key currently in this map.
- `lastKey()` - Returns the last (highest) key currently in this map.
- `headMap(K toKey)` - Returns a view of the portion of this map whose keys are strictly less than `toKey`.
- `tailMap(K fromKey)` - Returns a view of the portion of this map whose keys are greater than or equal to `fromKey`.
- `subMap(K fromKey, K toKey)` - Returns a view of the portion of this map whose keys range from `fromKey` (inclusive) to `toKey` (exclusive).
- `comparator()` - Returns the comparator used to order the keys, or null if natural ordering is used.

NavigableMap Interface (Extended SortedMap with navigation features)

- `lowerKey(K key)` - Returns the greatest key strictly less than the given key.
- `floorKey(K key)` - Returns the greatest key less than or equal to the given key.
- `ceilingKey(K key)` - Returns the least key greater than or equal to the given key.
- `higherKey(K key)` - Returns the least key strictly greater than the given key.
- `firstEntry()` / `lastEntry()` - Returns a key-value mapping associated with the least / greatest key.
- `pollFirstEntry()` / `pollLastEntry()` - Removes and returns the key-value mapping associated with the least / greatest key.
- `descendingMap()` - Returns a reverse-order view of the mappings contained in this map.

Iterator Interface

- `hasNext()` - Returns true if the iteration has more elements.
- `next()` - Returns the next element in the iteration.
- `remove()` - Removes from the underlying collection the last element returned by this iterator.
- `forEachRemaining(Consumer<? super E> action)` - Performs the given action for each remaining element until all elements have been processed.

ListIterator Interface (Bi-directional iterator for lists)

- `hasNext()` / `hasPrevious()` - Returns true if the list iterator has more elements when traversing forward / backward.
- `next()` / `previous()` - Returns the next / previous element in the list.
- `nextIndex()` / `previousIndex()` - Returns the index of the element that would be returned by a subsequent call to `next()` / `previous()`.

- `add(E e)` - Inserts the specified element into the list.
- `set(E e)` - Replaces the last element returned by `next()` or `previous()` with the specified element.
- `remove()` - Removes from the list the last element that was returned by `next()` or `previous()`.

🔧 System Requirements

Hardware

Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage

Software

Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal

1. SetExample.java — Set and NavigableSet

📄 Source Code

```
package collectionspractice.set;

import java.util.HashSet;
import java.util.NavigableSet;
import java.util.Set;
import java.util.TreeSet;

public class SetExample {
    public static void main(String[] args) {
        Set fruits = new HashSet<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Apple"); // A set keeps only one Apple.

        System.out.println("Unique fruits: " + fruits);
        System.out.println("Does it contain Apple? " + fruits.contains("Apple"));
        System.out.println("Number of unique fruits: " + fruits.size());
    }
}
```

```

fruits.remove("Banana");
System.out.println("After removing Banana: " + fruits);

NavigableSet numbers = new TreeSet<>();
// TreeSet keeps values in ascending order.
numbers.add(10);
numbers.add(20);
numbers.add(30);

System.out.println("Smallest number: " + numbers.first());
System.out.println("Largest number: " + numbers.last());
System.out.println("lower(20): " + numbers.lower(20));
System.out.println("floor(20): " + numbers.floor(20));
System.out.println("ceiling(25): " + numbers.ceiling(25));
System.out.println("higher(20): " + numbers.higher(20));
System.out.println("headSet(30): " + numbers.headSet(30));
System.out.println("tailSet(20): " + numbers.tailSet(20));
System.out.println("descendingSet: " + numbers.descendingSet());
System.out.println("pollFirst: " + numbers.pollFirst());
System.out.println("pollLast: " + numbers.pollLast());
System.out.println("remaining numbers: " + numbers);
}
}

```

Set is a collection that stores **unique elements only** — duplicates are silently ignored. `HashSet` stores elements in **no guaranteed order**. `TreeSet` (which implements `NavigableSet`) keeps elements **sorted in ascending order** and adds "navigation" methods to find neighbours of a value.

Method	Belongs to	What it does
<code>add()</code>	Set	Inserts an element; ignored if already present
<code>contains()</code>	Set	Checks if an element exists
<code>size()</code>	Set	Number of elements
<code>remove()</code>	Set	Deletes an element
<code>first()</code> / <code>last()</code>	<code>NavigableSet</code>	Smallest / largest element
<code>lower(x)</code>	<code>NavigableSet</code>	Largest element strictly less than x
<code>floor(x)</code>	<code>NavigableSet</code>	Largest element $\leq x$
<code>ceiling(x)</code>	<code>NavigableSet</code>	Smallest element $\geq x$
<code>higher(x)</code>	<code>NavigableSet</code>	Smallest element strictly greater than x
<code>headSet(x)</code>	<code>NavigableSet</code>	All elements less than x
<code>tailSet(x)</code>	<code>NavigableSet</code>	All elements greater than or equal to x
<code>descendingSet()</code>	<code>NavigableSet</code>	The set in reverse order
<code>pollFirst()</code> / <code>pollLast()</code>	<code>NavigableSet</code>	Removes and returns first/last element

Exception: No exception is thrown or caught in this program. A `TreeSet` **would** throw `NullPointerException` if you tried `add(null)`, because it needs to compare values to keep them sorted — this program never does that, so it runs cleanly.

2. Step-wise Execution

1. A `HashSet<String>` 'fruits' is created (raw type, no `<>` generic here).
2. "Apple" is added -> {Apple}
3. "Banana" is added -> {Apple, Banana} (order not guaranteed)
4. "Apple" is added again -> ignored, set stays {Apple, Banana}
5. Size printed = 2. `contains("Apple") = true`.
6. "Banana" is removed -> {Apple}
7. A `TreeSet<Integer>` 'numbers' is created as a `NavigableSet`.
8. 10, 20, 30 are added -> sorted internally as {10, 20, 30}
9. `first() = 10`, `last() = 30`
10. `lower(20) = 10` (strictly less than 20)
11. `floor(20) = 20` (less than or equal to 20)
12. `ceiling(25) = 30` (smallest ≥ 25)
13. `higher(20) = 30` (strictly greater than 20)
14. `headSet(30) = {10, 20}` (everything before 30)
15. `tailSet(20) = {20, 30}` (20 and everything after)
16. `descendingSet() = {30, 20, 10}`
17. `pollFirst()` removes and returns 10 -> set becomes {20, 30}
18. `pollLast()` removes and returns 30 -> set becomes {20}
19. remaining numbers printed = {20}

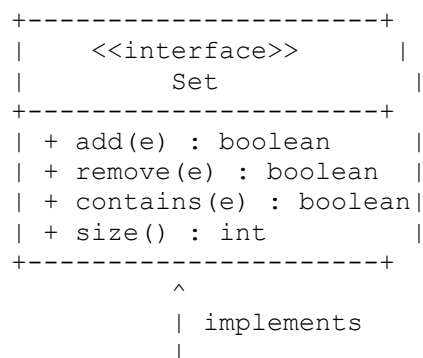
3. Algorithm

START

1. Create `HashSet` fruits
2. Add "Apple", "Banana", "Apple" (duplicate ignored)
3. Print set, `contains()`, `size()`
4. Remove "Banana"; print set
5. Create `TreeSet` numbers (as `NavigableSet`)
6. Add 10, 20, 30 (auto-sorted)
7. Print `first()`, `last()`
8. Print `lower(20)`, `floor(20)`, `ceiling(25)`, `higher(20)`
9. Print `headSet(30)`, `tailSet(20)`, `descendingSet()`
10. `pollFirst()` -> remove & print smallest
11. `pollLast()` -> remove & print largest
12. Print remaining set

STOP

4. UML Class Diagram





Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.set.setExample'
Unique fruits: [Apple, Banana]
Does it contain Apple? true
Number of unique fruits: 2
After removing Banana: [Apple]
Smallest number: 10
Largest number: 30
lower(20): 10
floor(20): 20
ceiling(25): 30
higher(20): 30
headSet(30): [10, 20]
tailSet(20): [20, 30]
descendingSet: [30, 20, 10]
pollFirst: 10
pollLast: 30
remaining numbers: [20]
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

2. QueueDequeExample.java — Queue and Deque

 Source Code

```

package collectionsppractice.queue;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Queue;

public class QueueDequeExample {

    public static void main(String[] args) {

        // Queue: First In, First Out (FIFO)
        Queue<String> queue = new ArrayDeque<>();

        queue.offer("First");
        queue.add("Second");

        System.out.println("People waiting: " + queue);
        System.out.println("Next person: " + queue.peek());
        System.out.println("Next person using element(): " +
queue.element());
        System.out.println("Served using poll(): " + queue.poll());
        System.out.println("Served using remove(): " + queue.remove());
        System.out.println("Nobody waiting? " + queue.isEmpty());

        // Deque: Allows changes at both ends
        Deque<String> deque = new ArrayDeque<>();

        deque.offerFirst("First");
        deque.offerLast("Final");
        deque.addFirst("Middle");
        deque.addLast("Last");

        System.out.println("Deque before removing: " + deque);
        System.out.println("peekFirst: " + deque.peekFirst());
        System.out.println("peekLast: " + deque.peekLast());
        System.out.println("pollFirst: " + deque.pollFirst());
        System.out.println("pollLast: " + deque.pollLast());
        System.out.println("removeFirst: " + deque.removeFirst());
        System.out.println("removeLast: " + deque.removeLast());
        System.out.println("Deque is empty? " + deque.isEmpty());
    }
}

```

Queue follows **FIFO (First In, First Out)** — like a line of people waiting. Deque (Double-Ended Queue) allows insertion and removal from **both ends**. Both are implemented here using ArrayDeque.

Method	Behaviour on empty queue	Belongs to
offer() / add()	insert; add() throws if capacity-restricted	Queue
peek()	returns null if empty	Queue
element()	throws NoSuchElementException if empty	Queue
poll()	returns null if empty	Queue

Method	Behaviour on empty queue	Belongs to
<code>remove()</code>	throws <code>NoSuchElementException</code> if empty	Queue
<code>offerFirst/offerLast</code>	insert at front/back	Deque
<code>peekFirst/peekLast</code>	return null if empty	Deque
<code>pollFirst/pollLast</code>	remove & return, null if empty	Deque
<code>removeFirst/removeLast</code>	throws <code>NoSuchElementException</code> if empty	Deque

2. Step-wise Execution

QUEUE (FIFO)

1. `ArrayDeque` used as a `Queue<String>` 'queue'.
2. `offer("First")` -> queue: [First]
3. `add("Second")` -> queue: [First, Second]
4. `peek()` = "First" (looks, does not remove)
5. `element()` = "First" (same as peek but throws if empty)
6. `poll()` removes & returns "First" -> queue: [Second]
7. `remove()` removes & returns "Second" -> queue: []
8. `isEmpty()` = true

DEQUE (both ends)

9. `ArrayDeque` used as `Deque<String>` 'deque'.
10. `offerFirst("First")` -> [First]
11. `offerLast("Final")` -> [First, Final]
12. `addFirst("Middle")` -> [Middle, First, Final]
13. `addLast("Last")` -> [Middle, First, Final, Last]
14. `peekFirst()` = "Middle", `peekLast()` = "Last"
15. `pollFirst()` removes "Middle" -> [First, Final, Last]
16. `pollLast()` removes "Last" -> [First, Final]
17. `removeFirst()` removes "First" -> [Final]
18. `removeLast()` removes "Final" -> []
19. `isEmpty()` = true

3. Algorithm

START

PART A – Queue

1. Create `ArrayDeque` as `Queue<String>`
2. `offer("First")`, `add("Second")`
3. Print queue, `peek()`, `element()`
4. `poll()` -> print and remove head
5. `remove()` -> print and remove new head
6. Print `isEmpty()`

PART B – Deque

7. Create `ArrayDeque` as `Deque<String>`
8. `offerFirst`, `offerLast`, `addFirst`, `addLast` to build deque
9. Print deque, `peekFirst()`, `peekLast()`
10. `pollFirst()` -> print and remove front

```

11. pollLast() -> print and remove back
12. removeFirst() -> print and remove new front
13. removeLast() -> print and remove new back
14. Print isEmpty()
STOP

```

4. UML Class Diagram



Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionpractice.queue.QueueDequeExample'
People waiting: [First, Second]
Next person: First
Next person using element(): First
Served using poll(): First
Served using remove(): Second
Nobody waiting? true
Deque before removing: [Middle, First, Final, Last]
peekFirst: Middle
peekLast: Last
pollFirst: Middle
pollLast: Last
removeFirst: First
removeLast: Final
Deque is empty? true

```

3. MapExample.java — Map and NavigableMap

Source Code

```
package collectionspractice.map;

import java.util.Map;
import java.util.NavigableMap;
import java.util.TreeMap;

public class MapExample {
    public static void main(String[] args) {
        Map students = new TreeMap<>();
        students.put(101, "Anu");
        students.put(102, "Bala");
        students.put(103, "Chitra");

        // Roll number is the key and the name is its value.
        System.out.println("Student list: " + students);
        System.out.println("Name for roll number 101: " + students.get(101));
        System.out.println("Is roll number 102 present? " +
            students.containsKey(102));
        System.out.println("Is Chitra present? " + students.containsValue("Chitra"));
        System.out.println("Only keys: " + students.keySet());
        System.out.println("Only values: " + students.values());
        System.out.println("Key and value together: " + students.entrySet());
        students.remove(103);
        System.out.println("After removing roll number 103: " + students);

        NavigableMap marks = new TreeMap<>();
        // TreeMap sorts entries by their keys.
        marks.put(60, "Pass");
        marks.put(80, "Good");
        marks.put(95, "Excellent");

        System.out.println("firstKey: " + marks.firstKey());
        System.out.println("lastKey: " + marks.lastKey());
        System.out.println("lowerKey(80): " + marks.lowerKey(80));
        System.out.println("floorKey(80): " + marks.floorKey(80));
        System.out.println("ceilingKey(85): " + marks.ceilingKey(85));
        System.out.println("higherKey(80): " + marks.higherKey(80));
        System.out.println("firstEntry: " + marks.firstEntry());
        System.out.println("lastEntry: " + marks.lastEntry());
        System.out.println("descendingMap: " + marks.descendingMap());
        System.out.println("pollFirstEntry: " + marks.pollFirstEntry());
        System.out.println("pollLastEntry: " + marks.pollLastEntry());
    }
}
```

Map stores data as **key-value pairs** — each key maps to exactly one value, and keys are unique. `TreeMap` (which implements `NavigableMap`) keeps entries **sorted by key** and adds navigation methods similar to `NavigableSet`, but working on keys and whole entries.

Method	Belongs to	What it does
<code>put(k,v)</code>	Map	Inserts/updates a key-value pair
<code>get(k)</code>	Map	Returns value for a key
<code>containsKey() / containsValue()</code>	Map	Checks presence
<code>keySet() / values() / entrySet()</code>	Map	Views of keys / values / pairs
<code>remove(k)</code>	Map	Deletes a pair by key
<code>firstKey() / lastKey()</code>	NavigableMap	Smallest / largest key
<code>lowerKey/floorKey/ceilingKey/higherKey</code>	NavigableMap	Neighbour keys (like NavigableSet)
<code>firstEntry() / lastEntry()</code>	NavigableMap	Smallest/largest whole entry
<code>descendingMap()</code>	NavigableMap	Map view in reverse key order
<code>pollFirstEntry() / pollLastEntry()</code>	NavigableMap	Removes & returns first/last entry

2. Step-wise Execution

1. TreeMap 'students' created as a Map (keys sorted automatically).
2. `put(101,"Anu"), put(102,"Bala"), put(103,"Chitra")`
-> {101=Anu, 102=Bala, 103=Chitra}
3. `get(101) = "Anu"`
4. `containsKey(102) = true`
5. `containsValue("Chitra") = true`
6. `keySet() = [101,102,103], values() = [Anu,Bala,Chitra]`
7. `entrySet() = [101=Anu, 102=Bala, 103=Chitra]`
8. `remove(103) -> {101=Anu, 102=Bala}`
9. TreeMap 'marks' created as NavigableMap.
10. `put(60,"Pass"), put(80,"Good"), put(95,"Excellent")`
-> sorted by key: {60=Pass, 80=Good, 95=Excellent}
11. `firstKey() = 60, lastKey() = 95`
12. `lowerKey(80) = 60, floorKey(80) = 80`
13. `ceilingKey(85) = 95, higherKey(80) = 95`
14. `firstEntry() = 60=Pass, lastEntry() = 95=Excellent`
15. `descendingMap() = {95=Excellent, 80=Good, 60=Pass}`
16. `pollFirstEntry()` removes & returns 60=Pass -> {80=Good, 95=Excellent}
17. `pollLastEntry()` removes & returns 95=Excellent -> {80=Good}

3. Algorithm

START

PART A – Map basics

1. Create TreeMap students
2. put three roll-number/name pairs
3. Print map, get(), containsKey(), containsValue()
4. Print keySet(), values(), entrySet()
5. remove(103); print map

PART B – NavigableMap

6. Create TreeMap marks (as NavigableMap)
7. put three mark/grade pairs
8. Print firstKey(), lastKey()
9. Print lowerKey(), floorKey(), ceilingKey(), higherKey()
10. Print firstEntry(), lastEntry(), descendingMap()
11. pollFirstEntry() -> print & remove smallest entry
12. pollLastEntry() -> print & remove largest entry

STOP

4. UML Class Diagram



Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetails
InExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.map.MapExample'
Student list: {101=Anu, 102=Bala, 103=Chitra}
Name for roll number 101: Anu
Is roll number 102 present? true
Is Chitra present? true
Only keys: [101, 102, 103]
Only values: [Anu, Bala, Chitra]
Key and value together: [101=Anu, 102=Bala, 103=Chitra]
After removing roll number 103: {101=Anu, 102=Bala}
firstKey: 60
lastKey: 95
lowerKey(80): 60
floorKey(80): 80
ceilingKey(85): 95
higherKey(80): 95
firstEntry: 60=Pass
lastEntry: 95=Excellent
descendingMap: {95=Excellent, 80=Good, 60=Pass}
pollFirstEntry: 60=Pass
pollLastEntry: 95=Excellent
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

4. VectorExample.java — Vector (legacy)

Source Code

```

package collectionspractice.list;

import java.util.Vector;
// legacy class

public class VectorExample {
    public static void main(String[] args) {
        Vector<Integer> numbers = new Vector<>();
        numbers.add(10);
        numbers.add(20);
        // Vector has a few older method names as well as normal List
    methods.
        numbers.addElement(30);
        numbers.insertElementAt(5, 0);

        System.out.println("Numbers in the vector: " + numbers);
        System.out.println("First number: " + numbers.firstElement());
        System.out.println("Last number: " + numbers.lastElement());
        System.out.println("Vector capacity: " + numbers.capacity());

        numbers.removeElement(Integer.valueOf(20));
        numbers.set(0, 1);
        System.out.println("Numbers after changes: " + numbers);
    }
}

```

`Vector` is a **legacy** (pre-Java-2) class that implements `List`. It behaves like `ArrayList` (a resizable array) but its methods are **synchronized** (thread-safe, though slower). It has both modern `List` methods (`add`, `set`) and old-style methods from before `List` existed (`addElement`, `insertElementAt`, `firstElement`, `lastElement`, `removeElement`).

Method	Style	What it does
<code>add(e)</code>	Modern (<code>List</code>)	Appends to the end
<code>addElement(e)</code>	Legacy	Same as <code>add()</code> , older name
<code>insertElementAt(e, i)</code>	Legacy	Inserts at a specific index
<code>firstElement()</code> / <code>lastElement()</code>	Legacy	Get first/last, throws <code>NoSuchElementException</code> if empty
<code>capacity()</code>	Legacy	Internal array size (not the element count)
<code>removeElement(e)</code>	Legacy	Removes first matching element
<code>set(i, e)</code>	Modern (<code>List</code>)	Replaces element at index

2. Step-wise Execution

```
1. Vector<Integer> numbers created.
2. add(10) -> [10]
3. add(20) -> [10, 20]
4. addElement(30) -> [10, 20, 30]
5. insertElementAt(5, 0) -> inserts 5 at index 0 -> [5, 10, 20, 30]
6. Print vector: [5, 10, 20, 30]
7. firstElement() = 5
8. lastElement() = 30
9. capacity() prints the vector's internal array size (>= 4, e.g. 10
   by default, doubling when it needs to grow)
10. removeElement(20) removes the value 20 -> [5, 10, 30]
11. set(0, 1) replaces index 0 with 1 -> [1, 10, 30]
12. Print vector after changes: [1, 10, 30]
```

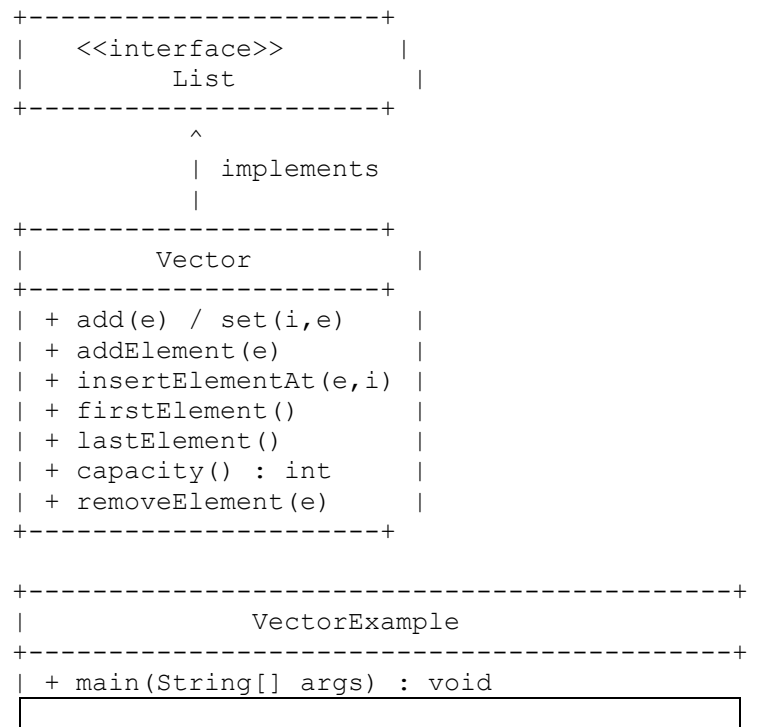
3. Algorithm

START

```
1. Create Vector<Integer> numbers
2. add(10), add(20)
3. addElement(30)
4. insertElementAt(5, 0)
5. Print vector
6. Print firstElement(), lastElement(), capacity()
7. removeElement(20)
8. set(0, 1)
9. Print vector after changes
```

STOP

4. UML Class Diagram



Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetails
InExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.list.VectorExample'
Numbers in the vector: [5, 10, 20, 30]
First number: 5
Last number: 30
Vector capacity: 10
Numbers after changes: [1, 10, 30]
PS C:\Users\Admin\OneDrive\Documents\basicjava>
```

5. StackExample.java — Stack (legacy)

Source Code

```
package collectionspractice.list;

import java.util.Stack;
// legacy class

public class StackExample {
    public static void main(String[] args) {
        Stack<String> books = new Stack<>();
```

```

        books.push("Java");
        books.push("SQL");
        books.push("Collections");

        // The last book placed on the stack is removed first.
        System.out.println("Books in the stack: " + books);
        System.out.println("Book on top: " + books.peek());
        System.out.println("SQL position from top: " + books.search("SQL"));
        System.out.println("Taking out: " + books.pop());
        System.out.println("Books left: " + books);
        System.out.println("Is the stack empty? " + books.empty());
    }
}

```

Stack is a **legacy class** that extends `Vector` and follows **LIFO (Last In, First Out)** — like a pile of books where you can only take from the top.

Method	What it does
<code>push(e)</code>	Adds an element to the top
<code>peek()</code>	Looks at the top element without removing it
<code>search(e)</code>	Returns 1-based position from the top, or -1 if not found
<code>pop()</code>	Removes and returns the top element
<code>empty()</code>	Checks if the stack has no elements

2. Step-wise Execution

1. `Stack<String> books` created.
2. `push("Java")` → [Java]
3. `push("SQL")` → [Java, SQL]
4. `push("Collections")` → [Java, SQL, Collections]
5. Print stack: [Java, SQL, Collections]
6. `peek()` = "Collections" (top, not removed)
7. `search("SQL")` = 2 (2nd from the top: Collections=1, SQL=2)
8. `pop()` removes & returns "Collections" → [Java, SQL]
9. Print stack left: [Java, SQL]
10. `empty()` = false (still has Java, SQL)

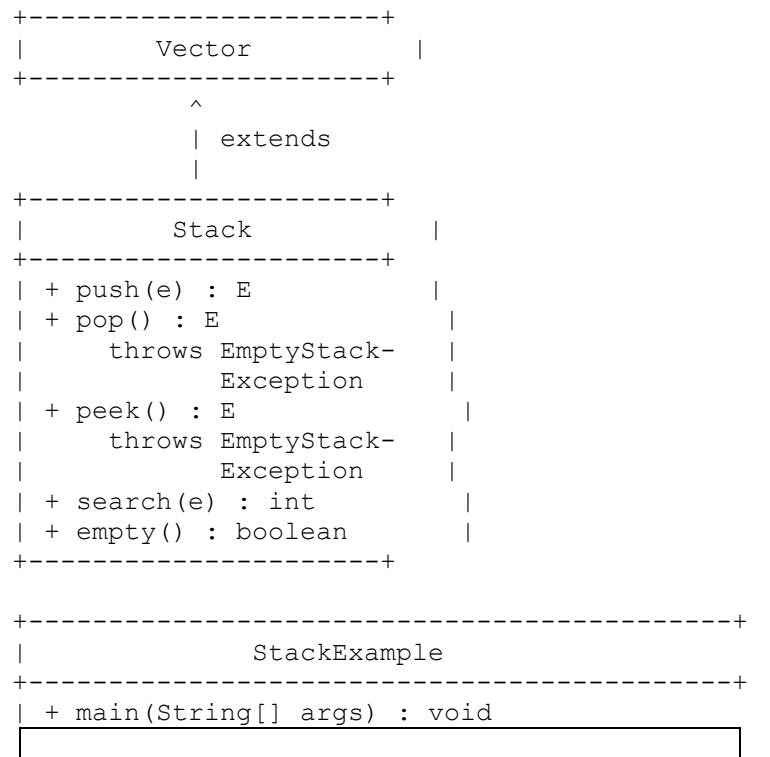
3. Algorithm

START

1. Create `Stack<String> books`
2. `push("Java"), push("SQL"), push("Collections")`
3. Print stack
4. `peek()` → print top element
5. `search("SQL")` → print position from top
6. `pop()` → print & remove top element
7. Print remaining stack
8. Print `empty()`

STOP

4. UML Class Diagram



Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetails
InExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.list.StackExample'
Books in the stack: [Java, SQL, Collections]
Book on top: Collections
SQL position from top: 2
Taking out: Collections
Books left: [Java, SQL]
Is the stack empty? false
PS C:\Users\Admin\OneDrive\Documents\basicjava>
```

6. LinkedListExample.java — LinkedList

Source Code

```
package collectionspractice.list;

import java.util.LinkedList;

public class LinkedListExample {
```

```

public static void main(String[] args) {

    LinkedList<String> tasks = new LinkedList<>();

    tasks.add("Study Java");
    tasks.addLast("Do exercise");
    tasks.addFirst("Wake up");
    tasks.offer("Go to sleep");

    System.out.println("Today's tasks: " + tasks);
    System.out.println("Second task: " + tasks.get(1));

    // Checking the first and last tasks
    System.out.println("First task: " + tasks.peek());
    System.out.println("Last task: " + tasks.peekLast());

    // Removing completed tasks
    System.out.println("Finished task: " + tasks.poll());
    System.out.println("Removed last task: " + tasks.removeLast());

    System.out.println("Tasks left: " + tasks);

    tasks.addFirst("Make a plan");
    tasks.addLast("Review the day");

    System.out.println("Removed first: " + tasks.removeFirst());
    System.out.println("Removed last: " + tasks.removeLast());

    System.out.println("Final tasks: " + tasks);
}
}

```

`LinkedList` implements **both List and Deque**, so it supports index-based access (`get()`) as well as fast insertion/removal from both ends (`addFirst`, `addLast`, `removeFirst`, `removeLast`). Internally, elements are linked nodes rather than a resizable array.

Method	What it does
<code>add(e)</code>	Appends to the end
<code>addFirst(e) / addLast(e)</code>	Inserts at front / back
<code>offer(e)</code>	Inserts at the end (Queue-style)
<code>get(i)</code>	Returns element at index <i>i</i>
<code>peek() / peekLast()</code>	Look at first / last without removing
<code>poll()</code>	Removes & returns the first element
<code>removeFirst() / removeLast()</code>	Removes & returns front / back

2. Step-wise Execution

1. `LinkedList<String> tasks` created.
2. `add("Study Java")` -> `[Study Java]`
3. `addLast("Do exercise")` -> `[Study Java, Do exercise]`

```

4.  addFirst("Wake up")    -> [Wake up, Study Java, Do exercise]
5.  offer("Go to sleep")   -> [Wake up, Study Java, Do exercise, Go to sleep]
6.  Print tasks: [Wake up, Study Java, Do exercise, Go to sleep]
7.  get(1) = "Study Java" (index 1)
8.  peek() = "Wake up" (first)
9.  peekLast() = "Go to sleep" (last)
10. poll() removes & returns "Wake up" -> [Study Java, Do exercise, Go to
sleep]
11. removeLast() removes & returns "Go to sleep" -> [Study Java, Do exercise]
12. Print tasks left: [Study Java, Do exercise]
13. addFirst("Make a plan") -> [Make a plan, Study Java, Do exercise]
14. addLast("Review the day") -> [Make a plan, Study Java, Do exercise,
Review the day]
15. removeFirst() removes & returns "Make a plan" -> [Study Java, Do
exercise, Review the day]
16. removeLast() removes & returns "Review the day" -> [Study Java, Do
exercise]
17. Print final tasks: [Study Java, Do exercise]

```

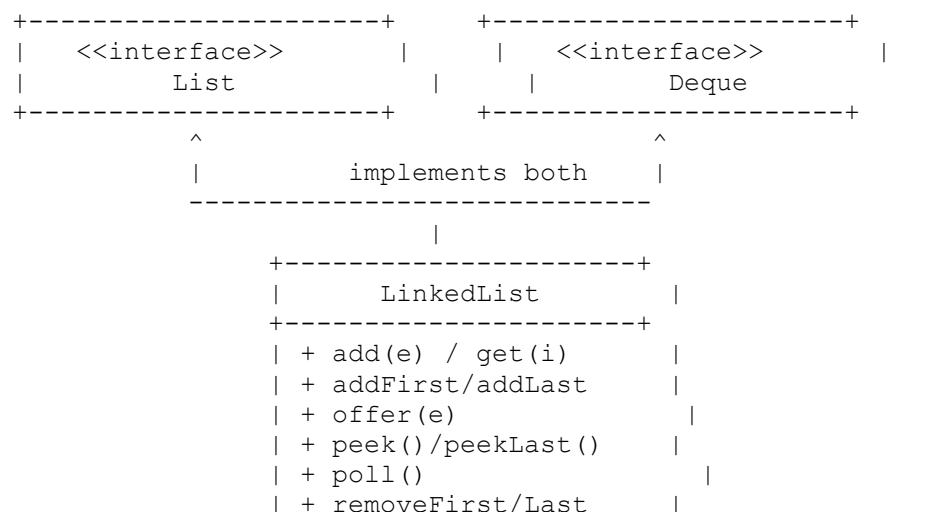
3. Algorithm

START

1. Create LinkedList<String> tasks
2. add, addLast, addFirst, offer to build the list
3. Print tasks; print get(1)
4. Print peek(), peekLast()
5. poll() -> print & remove first
6. removeLast() -> print & remove last
7. Print remaining tasks
8. addFirst("Make a plan"), addLast("Review the day")
9. removeFirst() -> print & remove new first
10. removeLast() -> print & remove new last
11. Print final tasks

STOP

4. UML Class Diagram



```

+-----+
|
|           LinkedListExample
|
+-----+
| + main(String[] args) : void
|
+-----+

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & C:\Program Files\Java\jdk-26.0.1\bin\java.exe --enable-preview -xx:+showCodeDetails
InExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.list.LinkedListExample'
Today's tasks: [Wake up, Study Java, Do exercise, Go to sleep]
Second task: Study Java
First task: Wake up
Last task: Go to sleep
Finished task: Wake up
Removed last task: Go to sleep
Tasks left: [Study Java, Do exercise]
Removed first: Make a plan
Removed last: Review the day
Final tasks: [Study Java, Do exercise]
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

7. ArrayListExample.java — ArrayList

Source Code

```

package collectionspractice.list;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Iterator;

public class ArrayListExample {

    public static void main(String[] args) {

        // Creating a list to store fruits
        List<String> fruits = new ArrayList<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add(1, "Mango");

        // Adding more fruits at once
        fruits.addAll(Arrays.asList("Orange", "Banana"));

        System.out.println("My fruits: " + fruits);
        System.out.println("Fruit at index 0: " + fruits.get(0));
        System.out.println("Is Apple present? " + fruits.contains("Apple"));
        System.out.println("Number of fruits: " + fruits.size());
    }
}

```

```

        System.out.println("First Banana index: " +
fruits.indexOf("Banana"));
        System.out.println("Last Banana index: " +
fruits.lastIndexOf("Banana"));

        // Changing and removing fruits
fruits.set(0, "Pear");
fruits.remove("Banana");
fruits.remove(0);

        // Sorting the fruits alphabetically
fruits.sort(String::compareTo);

System.out.println("First two fruits: " + fruits.subList(0, 2));
System.out.println("After changing the list: " + fruits);

        // Reading the fruits one by one using an Iterator
Iterator<String> iterator = fruits.iterator();

        while (iterator.hasNext()) {
            System.out.println("Fruit: " + iterator.next());
        }

        // Removing Orange from the list
fruits.removeAll(Arrays.asList("Orange"));
System.out.println("After removing Orange: " + fruits);

        // Removing all remaining fruits
fruits.clear();

        System.out.println("Is the list empty? " + fruits.isEmpty());
    }
}

```

`ArrayList` implements `List` and stores elements in a **resizable array**, keeping insertion order and allowing duplicates and index-based access. It's the most commonly used `List` implementation.

Method	What it does
<code>add(e) / add(i, e)</code>	Appends, or inserts at a specific index
<code>addAll(collection)</code>	Adds every element from another collection
<code>get(i)</code>	Returns element at index <code>i</code>
<code>contains() / size()</code>	Checks presence / counts elements
<code>indexOf() / lastIndexOf()</code>	First / last position of a value
<code>set(i, e)</code>	Replaces element at index <code>i</code>
<code>remove(Object) / remove(int)</code>	Removes by value or by index
<code>sort(Comparator)</code>	Sorts the list in place

Method	What it does
<code>subList(from, to)</code>	A view of part of the list
<code>iterator()</code>	Gets an <code>Iterator</code> to traverse the list
<code>removeAll(collection)</code>	Removes every matching element
<code>clear() / isEmpty()</code>	Empties the list / checks if empty

2. Step-wise Execution

```

1. ArrayList<String> fruits created (via List reference).
2. add("Apple") -> [Apple]
3. add("Banana") -> [Apple, Banana]
4. add(1,"Mango")-> [Apple, Mango, Banana]
5. addAll(["Orange","Banana"]) -> [Apple, Mango, Banana, Orange, Banana]
6. Print fruits: [Apple, Mango, Banana, Orange, Banana]
7. get(0) = "Apple"
8. contains("Apple") = true
9. size() = 5
10. indexOf("Banana") = 2 (first occurrence)
11. lastIndexOf("Banana") = 4 (last occurrence)
12. set(0,"Pear") -> [Pear, Mango, Banana, Orange, Banana]
13. remove("Banana") removes FIRST match -> [Pear, Mango, Orange, Banana]
14. remove(0) removes index 0 ("Pear") -> [Mango, Orange, Banana]
15. sort() alphabetically -> [Banana, Mango, Orange]
16. subList(0,2) = [Banana, Mango] (view, not removed from list)
17. Print list after changes: [Banana, Mango, Orange]
18. Iterator traverses and prints: Banana, Mango, Orange
19. removeAll(["Orange"]) -> [Banana, Mango]
20. Print after removing Orange: [Banana, Mango]
21. clear() -> []
22. isEmpty() = true

```

3. Algorithm

START

```

1. Create ArrayList fruits (as List reference)
2. add, add(index,e), addAll to build the list
3. Print list, get(0), contains(), size()
4. Print indexOf(), lastIndexOf()
5. set(0, "Pear")
6. remove(Object "Banana"); remove(index 0)
7. sort(String::compareTo)
8. Print subList(0,2); print full list
9. Get Iterator; WHILE hasNext(): print next()
10. removeAll(["Orange"]); print list
11. clear(); print isEmpty()

```

STOP

4. UML Class Diagram

+-----+



Output:

8. IteratorExample.java — Iterator and ListIterator

Source Code

```
package collectionspractice.iterator;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.ListIterator;

public class IteratorExample {
    public static void main(String[] args) {
        List<String> subjects = new ArrayList<>();
        subjects.add("Java");
        subjects.add("SQL");
    }
}
```

```

subjects.add("HTML");

// removes the current item
Iterator<String> iterator = subjects.iterator();
while (iterator.hasNext()) {
    String subject = iterator.next();
    System.out.println("Reading: " + subject);
    if (subject.equals("HTML")) {
        iterator.remove();
    }
}
System.out.println("Subjects after removing HTML: " + subjects);

ListIterator<String> listIterator = subjects.listIterator();
while (listIterator.hasNext()) {
    System.out.println("Reading from index: " +
listIterator.nextIndex());
    listIterator.next();
}
// ListIterator can move backwards and forward
while (listIterator.hasPrevious()) {
    String subject = listIterator.previous();
    System.out.println("Going backwards: " + subject);
    if (subject.equals("SQL")) {
        listIterator.set("Database");
    }
}
listIterator.add("Programming");

System.out.println("Final subjects: " + subjects);
System.out.println("Current previous index: " +
listIterator.previousIndex());
}
}

```

Iterator lets you traverse a collection **one element at a time, forward only**, and it's the **safe** way to remove elements *while* looping (directly calling `list.remove()` inside a for-each loop throws `ConcurrentModificationException`). `ListIterator` extends `Iterator` and adds **backward traversal**, plus `set()` (replace the last returned element) and `add()` (insert during traversal).

Method	Belongs to	What it does
<code>hasNext()</code> / <code>next()</code>	Iterator	Checks / moves forward
<code>remove()</code>	Iterator	Removes the last element returned by <code>next()</code>
<code>nextIndex()</code>	ListIterator	Index that the next <code>next()</code> call would return
<code>hasPrevious()</code> / <code>previous()</code>	ListIterator	Checks / moves backward
<code>set(e)</code>	ListIterator	Replaces the last element returned
<code>add(e)</code>	ListIterator	Inserts a new element at the current position

Method	Belongs to	What it does
<code>previousIndex()</code>	<code>ListIterator</code>	Index that the next <code>previous()</code> call would return

2. Step-wise Execution

1. `ArrayList<String> subjects: add "Java","SQL","HTML"`
`-> [Java, SQL, HTML]`
2. Get Iterator; loop while `hasNext()`:
`next() = "Java" -> print "Reading: Java"`
`next() = "SQL" -> print "Reading: SQL"`
`next() = "HTML" -> print "Reading: HTML"`
`-> equals "HTML" -> iterator.remove()`
`-> subjects becomes [Java, SQL]`
3. Print subjects after removing HTML: `[Java, SQL]`
4. Get `ListIterator`; loop forward while `hasNext()`:
`nextIndex()=0 -> next() moves past "Java"`
`nextIndex()=1 -> next() moves past "SQL"`
(cursor now sits AFTER "SQL", at the end of the list)
5. Loop backward while `hasPrevious()`:
`previous() = "SQL" -> print "Going backwards: SQL"`
`-> equals "SQL" -> listIterator.set("Database")`
`-> subjects becomes [Java, Database]`
`previous() = "Java" -> print "Going backwards: Java"`
6. `listIterator.add("Programming")` inserts at the CURRENT cursor position (before "Java", since cursor is now at the start)
`-> subjects becomes [Programming, Java, Database]`
7. Print final subjects: `[Programming, Java, Database]`
8. `previousIndex() = 0` (index just before the cursor, which now sits right after the newly added "Programming")

3. Algorithm

START

1. Create `ArrayList` subjects; add "Java", "SQL", "HTML"
2. Get Iterator from subjects
3. WHILE `iterator.hasNext()`:
`subject = iterator.next(); print it`
IF subject equals "HTML": `iterator.remove()`
4. Print subjects after removing HTML
5. Get `ListIterator` from subjects
6. WHILE `listIterator.hasNext()`:
`print nextIndex(); call next() to move forward`
7. WHILE `listIterator.hasPrevious()`:
`subject = listIterator.previous(); print it`
IF subject equals "SQL": `listIterator.set("Database")`
8. `listIterator.add("Programming")`
9. Print final subjects
10. Print `listIterator.previousIndex()`

STOP

4. UML Class Diagram

```

+-----+
| <<interface>> |
| Iterator |
+-----+
| + hasNext() : boolean |
| + next() : E |
| + remove() : void |
+-----+
      ^
      | extends
      |
+-----+
| <<interface>> |
| ListIterator |
+-----+
| + hasPrevious() / |
| previous() |
| + nextIndex() / |
| previousIndex() |
| + set(e) / add(e) |
+-----+

+-----+
| IteratorExample |
+-----+
| + main(String[] args) : void |
+-----+
      | uses
      v
ArrayList (provides the Iterator / ListIterator)

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & "C:\Program Files\Java\jdk-20.0.1\bin\java.exe" --enable-preview -XX:+ShowCodeDetails -ea InExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.list.ArrayListExample'
My fruits: [Apple, Mango, Banana, Orange, Banana]
Fruit at index 0: Apple
Is Apple present? true
Number of fruits: 5
First Banana index: 2
Last Banana index: 4
First two fruits: [Banana, Mango]
After changing the list: [Banana, Mango, Orange]
Fruit: Banana
Fruit: Mango
Fruit: Orange
After removing Orange: [Banana, Mango]
Is the list empty? true
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

Result

The Java Collection Framework was successfully explored by implementing eight programs covering its key interfaces and classes — Set (HashSet, TreeSet/NavigableSet), Queue and

Deque (ArrayDeque), Map (TreeMap/NavigableMap), the legacy classes Vector and Stack, the List implementations LinkedList and ArrayList, and traversal using Iterator and ListIterator. Each program correctly demonstrated the important methods of its interface — adding, removing, searching, sorting, and navigating elements — confirming that the Collection Framework provides efficient, ready-made data structures for storing and manipulating groups of objects in Java.